CITS5501/CITS3501 Project 2026 Phase 2

1. Changes

– Version 0.2
– Corrected typos; no report min or max

2. Project admin

Due to the short time available, phase 2 will be marked out of **15**; so we have the individual component (5 marks), phase 1 (5 marks), and phase 2 (15 marks). This gives a total of 25 marks; the grade for the project is still 35%.

Due date: 11:59 pm, Sun 18 Oct.

The 48-hour "waiver period" can be made use of if needed.

Four tasks are described, so that 1 person should be able to complete one task this week (though they might involve discussions with others), and an additional person can compile and coordinate content.

2.1. Format and submission of your work

As with phase 1, all code and reports should be submitted by making a Moodle submission.

There are no minimum or maximum lengths for report sections, but concision is valued – we recommend at most 1-2 pages per section.

3. Phase 2 activities

3.1. Structure

You will need to write up a short report and submit some code.

An outline of the tasks is as follows.

Task	Marks	Purpose	Deliverables
Task 1 – Prototype	4	Minimal prototype scraping code and brief documentation	Code, discussion in report
Task 2 – Test design	4	Specify tests for given failure scenarios	Test-case table and short discussion
Task 3 – Fixtures and doubles	4	Explain and demonstrate how tests can be made repeatable	Short discussion, one artifact
Task 4 – Adversarial inputs	3	Identify further assumptions and devise tests to expose them	Table of test ideas

3.2. Task 1 – Prototype

Implement a *minimal* prototype, which aims to identify and download policy documents from the site https://uwa-policy.cits5501.arranstewart.io/policy/. *Policy documents* here means documents the site links to, which can be opened by typical office programs or PDF viewers. You will briefly discuss your prototype in a report section ("Task 1 – Prototype").

The requirements are:

1. Start from the supplied policy index page. There is no *need* to handle other starting pages (though you may wish to do so for your own testing purposes – e.g. allow http://localhost:3000 as a starting page).
2. Discover a selection of links to potentially downloadable policy documents.
   - You do *not* need to include links to other websites, links which don't reasonably appear to lead to a policy document, or webpage assets intended to be displayed inline (like Javascript files or images).
   - You do *not* need to do any sort of recursive crawling – you may assume that all policy documents are accessible from the index page, and that you do not need to process any links from within those documents.
   - When deciding whether something leads to a policy document – filtering by file extension of the linked file (e.g. .pdf, .docx) is acceptable. (But you may wish to discuss the limitations of this in later tasks.)
3. Attempt to download a small number (3-5) of distinct documents, subject to reasonable request limits (e.g. make no more than 20 requests total; give up if they do not succeed). You need not keep the downloaded file on disk – it's enough to print out the download was successful (or not), and the metadata (see next item), then delete (or just discard) any downloaded data.
4. Display enough metadata to evaluate the outcome: for instance, (a) the policy URL, (b) HTTP status,

   (c) declared content type, (d) downloaded size, and (e) (optional) inital bytes of the downloaded file.

   Metadata item (e) is included because modern office files often will start with the bytes `PK`, and PDF files will start with the bytes `%PDF`. This does not fully guarantee that the file is a usable file of some type, but is a good start.

Write a small Python or Java program that implements the above requirements. Starter implementations are provided on Moodle. The prototype does *not* need to be robust (e.g. handling server timeouts or unsuccessful responses) – it is just a very minimal proof of concept.

Submit the code and your report section. The report section should explain what the prototype does, and at least 2 limitations of the prototype. The requirements are deliberately small in scope; briefly document any assumptions you need to make in order to keep things achievable.

Your code should be submitted as a zip file, and should include a README.md explaining how to compile and run it.

3.3. Task 2 – Design of test suite

Suppose you wanted to implement a full test suite, for a fully functional scraper. For task 2, we ask you to identify a minimum, credible test suite for a production scraper. A scraper typically only works against *one specific site* (or a small number of known sites), and the tests have to allow for reasonable behaviour when the site is misconfigured, unavailable, or otherwise behaves unusually.

The aim for this task is for you to identify reasonable expectations for a scraper and design tests for the supplied scenarios given below. These scenarios represent behaviours that a well-engineered scraper should handle predictably (the appropriate response might be simply to report a failure rather than recover).

Test scenarios

1. The structure of the website changes, such that your original scraping techniques no longer apply.
2. The HTML seems to be truncated.
3. The site, or a particular link on it, is unavailable (times out), either for a short or long period.
4. The site, or particular links on it, return status codes in the 400-499 or 500-599 range. (These imply there is something wrong with the client request, in the former case, or the server's operation, in the latter.)
5. A link to a policy document does not return a well-formed office or PDF file.
   Example 1: the link is to a PDF file, but the server returns HTML content.
   Example 2: the link is to a PDF file, but the server returns a corrupt file which cannot be opened by a PDF viewer.

In your report, address these scenarios, explaining:

– What is a reasonable expectation for what a scraper should do in this scenario?

– Provide a table or list of *test cases* addressing the scenarios – at least one test case per scenario, roughly 6–10 in total.

  Give each test case a unique ID (e.g. T001, T002, etc.), and specify which scenario it relates to. Include brief details so your test could be implemented by teaching staff or another student. Would you need to do anything to set up the test case ("Arrange")? What would count as evidence the scraper is behaving correctly or incorrectly ("Assert")? (Usually, the "Act" part of the test case will be straightforward, but you can document that as well if needed.)

3.4. Task 3 – Fixtures and test doubles

This has two components – a brief discussion of fixtures and test doubles, and then a sample code/ artifact deliverable (plus an explanation of it in your report).

Brief discussion

Explain in your report how tests for the Task 2 scenarios could make use of fixtures and test doubles. Specifically:

1. What kind of fixtures would you use as a "source of truth" for whether your scraper is working correctly? (For instance, HTML files, policy files, locally run servers)
2. Does the target site seem to limit high levels of requests? Describe your observations. You do not need to use your prototype scraper – instead, it should be enough to use a tool like curl, which is available for all common platforms, to probe the site. (It should be possible to identify limits with no more than 10 attempted downloads; but if you cannot identify any, just document what you tried and what your observations were.) If requests to the actual site are limited, how would you create and run your tests from Task 2 (Design of test suite)?
3. Can test doubles be used to simulate timeouts, interrupted downloads, connection failures, or very slow responses? What tools or frameworks are available in your chosen language for doing this?

You do not need to refer to exact test IDs from Task 2 – thus, Tasks 2 and 3 can be done in parallel (though we recommend someone read the report sections after they are done to ensure they are at least generally consistent).

A good approach is if Task 2 contains at most *brief* mentions of any fixtures and test doubles needed; task 3 can contain a more detailed discussion.

Code/artifact deliverable

Demonstrate one failure scenario using a fixture or test double. In other words, provide a fixture or double that provides an example of how the scraper might fail a test. The fixture or test double can be any artifact of your choosing – it could be a policy file, a fragment of HTML, a script which brings up a local server (but with some sort of demonstrable problem); any of those would be suitable options here.

Explain in your report why the same scenario is difficult or undesirable to reproduce against the actual live site.

3.5. Task 4 – Adversarial inputs

Suppose you were testing some *other* team's scraper. How would you construct unusual inputs that expose assumptions the team has made about how a scraper's environment "should work"?

(A suggested approach is to brainstorm in a group: try to make as complete a list as you can of how a "normal" or "happy path" request works – and then think of environment setups or inputs that would expose potential problems.)

Try to identify 5-8 ideas, and put these in a table or list. Each idea should state the following: (a) what is the assumption being challenged/tested, (b) what input artifact or environment would be used to test this, (c) what would count as "failure", and how would that be detected?

3.6. Submission

Submit the markdown and pdf versions of your report, and a zip file of code, in the CITS5501/3510 Project submission area in Moodle.

One student only per group should submit the project.

The report should include links to your team's private github repository for the project.

At the start of your report, list all members of the group and their

– student number
– GitHub/GitLab/other user ID.

Please number the answers in your report. Remember to ensure your facilitator has access to your repository.

A submission *checklist* for your use:

1. Zip file. Should contain:
   – Any code for your Task 1 minimal prototype
   – A README.md file explaining how to run the prototype
   – Any fixtures or test double code from Task 3
2. Report. Submitted as both PDF and markdown. Should contain any sections/write-up for Task 1, 2, 3 and 4.
3. Git repository. Not submitted, but your report should provide a link to it.

Marking will normally be based on the PDF version of the report, but if the marker has trouble reading it, they may refer to your markdown version.